

对于python中的正则表达式理解不深，请给出保姆级的详细教程--比如举例子，使用图片等等易于理解、便于记忆的方式。另，同样再详细将以下re库



Python正则表达式与re库 保姆级教程

第一部分：正则表达式基础

1. 什么是正则表达式？

正则表达式（Regular Expression）是一种用于匹配字符串模式的强大工具。可以把它想象成一个"字符串模板"，用来在文本中查找、替换符合特定规则的字符串。

2. 基本概念图解

text

复制 下载

文本："我的电话是138-1234-5678，你的电话是139-8765-4321"

正则表达式：\d{3}-\d{4}-\d{4}

匹配结果：138-1234-5678 和 139-8765-4321

[https://via.placeholder.com/600x200?](https://via.placeholder.com/600x200?text=%E6%AD%A3%E5%88%99%E8%A1%A8%E8%BE%BE%E5%BC%8F+%E5%8C%B9%E9%85%8D+%E7%97%B5%E8%A4%B2%E5%B8%B7%E7%A0%81+)

[text=%E6%AD%A3%E5%88%99%E8%A1%A8%E8%BE%BE%E5%BC%8F+%E5%8C%B9%E9%85%8D+%E7%97%B5%E8%A4%B2%E5%B8%B7%E7%A0%81+](#)

3. 元字符详解

3.1 字符匹配

元字符	说明	示例	匹配结果
.	匹配任意单个字符	a.b	"aab", "acb", "a1b"
\d	匹配数字	\d\d	"12", "45", "99"
\D	匹配非数字	\D\D	"ab", "你好"
\w	匹配字母、数字、下划线	\w\w	"ab", "12", "a1"
\W	匹配非字母数字下划线	\W\W	"!!", " "
\s	匹配空白字符	a\s b	"a b", "a\tb"
\S	匹配非空白字符	a\S b	"a1b", "a-b"

3.2 数量限定符

限定符	说明	示例	匹配结果
*	0次或多次	ab*c	"ac", "abc", "abbc"
+	1次或多次	ab+c	"abc", "abbc"
?	0次或1次	ab?c	"ac", "abc"
{n}	恰好n次	a{3}	"aaa"
{n,}	至少n次	a{2,}	"aa", "aaa"
{n,m}	n到m次	a{2,4}	"aa", "aaa", "aaaa"

3.3 位置锚点

锚点	说明	示例	匹配结果
^	字符串开始	^abc	"abc"开头的字符串
\$	字符串结束	xyz\$	"xyz"结尾的字符串
\b	单词边界	\bcat\b	"cat"单词

3.4 字符类

表达式	说明	示例	匹配结果
<code>[abc]</code>	匹配a,b,c任意一个	<code>[abc]123</code>	"a123", "b123", "c123"
<code>[a-z]</code>	匹配a到z任意字母	<code>[a-z]+</code>	"hello", "world"
<code>[^abc]</code>	匹配除了a,b,c的字符	<code>[^abc]123</code>	"d123", "1123"

4. 分组和选择

表达式	说明	示例	匹配结果
<code>()</code>	分组	<code>(abc)+</code>	"abc", "abcabc"
<code>,</code>	<code>,</code>	或操作	<code>`cat`</code> <code>dog`</code> <code>"c"</code>
<code>(?:)</code>	非捕获分组	<code>(?:abc)+</code>	"abc", "abcabc"

第二部分：Python re库详解

1. re库常用函数

1.1 re.match() - 从字符串开头匹配

python

 复制  下载

```
import re

# 从字符串开头匹配
pattern = r"hello"
text = "hello world"

result = re.match(pattern, text)
if result:
    print("匹配成功:", result.group()) # 输出: hello
else:
    print("匹配失败")

# 匹配失败示例
result2 = re.match(r"world", "hello world")
print(result2) # None
```

1.2 re.search() - 搜索整个字符串

python

 复制  下载

```
import re

# 在字符串中搜索匹配
pattern = r"world"
text = "hello world, welcome to the world"

result = re.search(pattern, text)
if result:
    print("找到匹配:", result.group()) # 输出: world
    print("匹配位置:", result.span()) # 输出: (6, 11)
```

1.3 re.findall() - 查找所有匹配

python

 复制  下载

```
import re

# 查找所有匹配的字符串
pattern = r"\d+" # 匹配一个或多个数字
text = "我有3个苹果, 2个香蕉, 15颗葡萄"

results = re.findall(pattern, text)
print(results) # 输出: ['3', '2', '15']
```

1.4 re.finditer() - 返回迭代器

python

 复制  下载

```
import re

# 返回匹配对象的迭代器
pattern = r"\d+"
text = "我有3个苹果, 2个香蕉, 15颗葡萄"

for match in re.finditer(pattern, text):
    print(f"找到数字: {match.group()}, 位置: {match.span()}")
# 输出:
# 找到数字: 3, 位置: (2, 3)
# 找到数字: 2, 位置: (8, 9)
# 找到数字: 15, 位置: (13, 15)
```

1.5 re.sub() - 替换匹配内容

python

 复制  下载

```
import re

# 替换匹配的文本
pattern = r"苹果"
text = "我喜欢吃苹果, 苹果很甜"

# 替换所有匹配
new_text = re.sub(pattern, "香蕉", text)
print(new_text) # 输出: 我喜欢吃香蕉, 香蕉很甜

# 使用函数进行替换
def to_upper(match):
    return match.group().upper()

text2 = "hello world"
new_text2 = re.sub(r"\b\w+\b", to_upper, text2)
print(new_text2) # 输出: HELLO WORLD
```

1.6 re.split() - 根据模式分割字符串

python

 复制  下载

```
import re

# 使用正则表达式分割字符串
text = "苹果,香蕉;葡萄 橙子|梨"

# 使用多种分隔符分割
result = re.split(r"[;, \|]", text)
print(result) # 输出: ['苹果', '香蕉', '葡萄', '橙子', '梨']

# 复杂的分割示例
text2 = "单词123和456数字"
result2 = re.split(r"\d+", text2)
print(result2) # 输出: ['单词', '和', '数字']
```

2. 匹配对象的方法

python

 复制  下载

```
import re

text = "我的电话是: 138-1234-5678"
pattern = r"(\d{3})-(\d{4})-(\d{4})"

match = re.search(pattern, text)
if match:
    print("完整匹配:", match.group())          # 138-1234-5678
    print("分组1:", match.group(1))            # 138
    print("分组2:", match.group(2))            # 1234
    print("分组3:", match.group(3))            # 5678
    print("所有分组:", match.groups())         # ('138', '1234', '5678')
    print("匹配位置:", match.span())           # (6, 19)
    print("分组1位置:", match.span(1))         # (6, 9)
```

3. 编译正则表达式

对于需要重复使用的正则表达式，可以先编译以提高效率：

python

 复制  下载

```
import re

# 编译正则表达式
pattern = re.compile(r"\d{3}-\d{4}-\d{4}")

text = "电话1: 138-1234-5678, 电话2: 139-8765-4321"

# 使用编译后的模式对象
result1 = pattern.search(text)
print(result1.group()) # 138-1234-5678

result2 = pattern.findall(text)
print(result2) # ['138-1234-5678', '139-8765-4321']
```

4. 标志参数 (flags)

python

 复制

 下载

```
import re

text = "Hello\nWORLD\npython"

# IGNORECASE - 忽略大小写
result1 = re.findall(r"hello", text, re.IGNORECASE)
print(result1) # ['Hello']

# MULTILINE - 多行模式
result2 = re.findall(r"^.+$", text, re.MULTILINE)
print(result2) # ['Hello', 'WORLD', 'python']

# DOTALL - 让.匹配包括换行符的所有字符
result3 = re.findall(r"Hello.*python", text, re.DOTALL)
print(result3) # ['Hello\nWORLD\npython']

# VERBOSE - 允许在正则表达式中添加注释和空白
pattern = re.compile(r"""
    \d{3}    # 前3位数字
    -       # 分隔符
    \d{4}    # 中间4位数字
    -       # 分隔符
    \d{4}    # 后4位数字
""", re.VERBOSE)
```

第三部分：实战案例

案例1：验证邮箱格式

python

 复制  下载

```
import re

def validate_email(email):
    pattern = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'
    return bool(re.match(pattern, email))

# 测试
emails = [
    "user@example.com",
    "user.name@domain.co.uk",
    "invalid-email",
    "user@domain",
    "user@.com"
]

for email in emails:
    print(f"{email}: {'有效' if validate_email(email) else '无效'}")
```

案例2：提取网页中的链接

python

 复制  下载

```
import re

def extract_links(html):
    # 匹配href属性中的链接
    pattern = r'href="([^"]*)"'
    return re.findall(pattern, html)

# 示例HTML
html = '''
<a href="https://www.example.com">首页</a>
<a href="/about">关于我们</a>
<a href="contact.html">联系我们</a>
'''

links = extract_links(html)
print("提取到的链接:", links)
```

案例3：数据清洗

python

 复制  下载

```
import re

def clean_text(text):
    # 移除多余的空格
    text = re.sub(r'\s+', ' ', text)
    # 移除特殊字符，只保留中文、英文、数字和常用标点
    text = re.sub(r'^\u4e00-\u9fa5a-zA-Z0-9\s.,!?]', '', text)
    # 移除首尾空格
    text = text.strip()
    return text

dirty_text = " 这是一段 有问题的文本!! @#$% 包含特殊字符。。。 "
clean = clean_text(dirty_text)
print("清洗前:", repr(dirty_text))
print("清洗后:", repr(clean))
```

第四部分：调试技巧

1. 使用在线工具调试

推荐使用以下在线正则表达式测试工具：

[Regex101](#)

[RegExr](#)

2. 逐步构建正则表达式

python

复制 下载

```
import re

text = "姓名：张三，年龄：25，邮箱：zhangsan@example.com"

# 逐步构建复杂的正则表达式
# 步骤1：匹配姓名
name_pattern = r"姓名:\s*([^\,]+)"
name_match = re.search(name_pattern, text)
if name_match:
    print("姓名:", name_match.group(1))

# 步骤2：匹配年龄
age_pattern = r"年龄:\s*(\d+)"
age_match = re.search(age_pattern, text)
if age_match:
    print("年龄:", age_match.group(1))

# 步骤3：匹配邮箱
email_pattern = r"邮箱:\s*([^\,]+)"
email_match = re.search(email_pattern, text)
if email_match:
    print("邮箱:", email_match.group(1))
```

总结

概念	说明	示例
基本匹配	使用元字符匹配特定模式	<code>\d</code> 匹配数字
数量限定	控制匹配字符的数量	<code>a{2,4}</code> 匹配2-4个a
分组	将模式分组以便引用	<code>(abc)+</code> 匹配多个abc
re函数	Python中的正则操作函数	<code>re.findall()</code> 查找所有匹配
编译	提高重复使用效率	<code>re.compile()</code>
标志	修改匹配行为	<code>re.IGNORECASE</code> 忽略大小写

记住：正则表达式需要多练习才能熟练掌握。建议从简单的模式开始，逐步构建复杂的表达式，并使用在线工具进行测试和调试。

本回答由 AI 生成，内容仅供参考，请仔细甄别。